Nume și prenume: Chirițoiu Constantin-Sorin

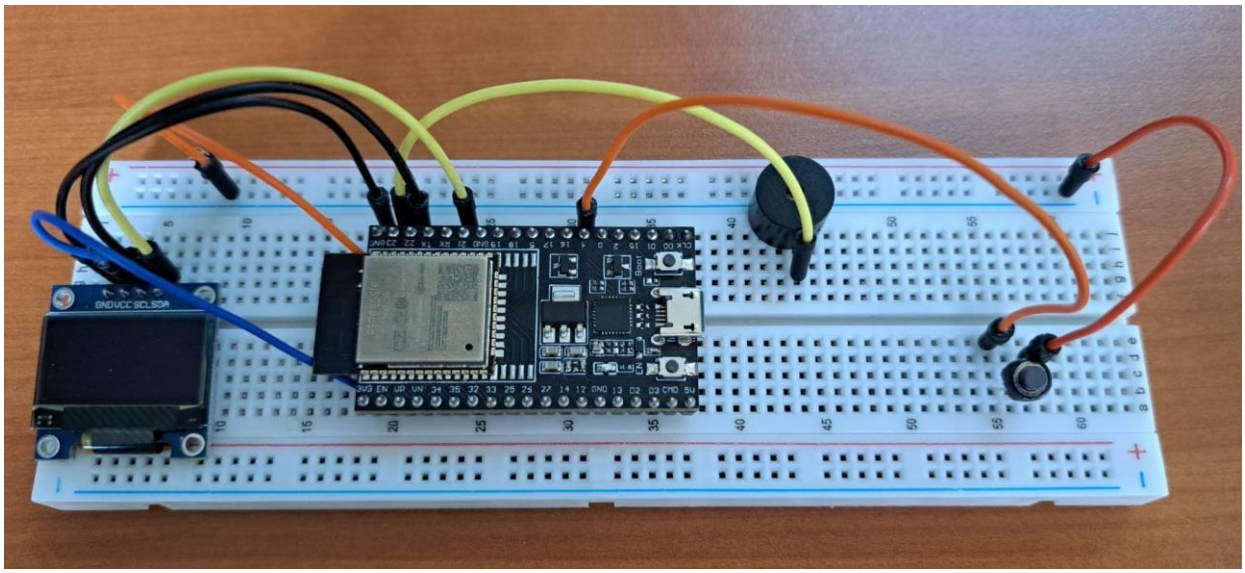
Mini-proiect: **Flappy Bird**

1. Materiale folosite

-Breadboard
-ESP32
-Modul afișaj SSD1306
-Buzzer
-Buton
-8 cabluri

2. Modul de lucru

Montaj [1]



Conectarea pieselor cu pinii microcontroller-ului se face astfel:

- Pentru afișajul SSD1306:
  -GND -> GND
  -VCC -> 3V3
  -SCL -> D22
  -SDA -> D21

- Pentru Buzzer: -GND
  -> GND
  -Pol pozitiv -> D23

- Pentru buton: -Pol -
  > GND
  -Pol -> D4

Cod [2]

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define BUTTON_PIN 4
#define BUZZER_PIN 23

#define BIRD_WIDTH 8
#define BIRD_HEIGHT 8
 const uint8_t bird_bitmap[] PROGMEM =
{
  0b00001110,
  0b00011111,
  0b00111111,
  0b01111111,
  0b11111111,
  0b00111000,
  0b01110100,
  0b00001010
};

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
 int birdY; float
birdVelocityY; int
gravity = 1; bool
gameStarted = false;
bool gameOver = false;
int obstacleX; int
obstacleGap = 20; int
obstacleWidth = 10; int
obstacleSpeed = 2;
```

```
int score = 0;

#define NOTE_C4 262
#define NOTE_E3 165
#define NOTE_G3 196
#define NOTE_E4 330
#define NOTE_G4 392

int melody[] = {
  NOTE_C4, NOTE_E3, NOTE_G3, NOTE_E4,
  NOTE_G4, NOTE_C4, NOTE_E4,
NOTE_G4 };  int noteDurations[] = {
  4, 8, 8, 4,
  4, 4, 4, 2
};  int currentNote = 0;
unsigned long noteStartTime =
0;

void setup() {   Wire.begin();   if
(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
for (;;);
  }    display.display();
delay(2000);
display.clearDisplay();
display.display();
pinMode(BUTTON_PIN, INPUT_PULLUP);
birdY = SCREEN_HEIGHT / 2;
birdVelocityY = 0;   obstacleX =
SCREEN_WIDTH;   pinMode(BUZZER_PIN,
OUTPUT);
}
void loop() {   if
(!gameStarted) {
display.clearDisplay();
display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
display.setCursor(10, 20);
display.println("Flappy Bird");
display.setCursor(5, 45);     display.println("Press
to start");     display.display();
```

```
     if (digitalRead(BUTTON_PIN) == LOW)
{        gameStarted = true;
    }
return;
  }    if (gameOver) {
display.clearDisplay();
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(10, 20);
display.println("Game Over");
display.setCursor(5, 45);
display.println("Press to play again");
display.display();
     if (millis() - noteStartTime >= noteDurations[currentNote])
{        noTone(BUZZER_PIN);       currentNote++;       if
(currentNote < sizeof(melody) / sizeof(melody[0])) {
tone(BUZZER_PIN, melody[currentNote]);         noteStartTime =
millis();
      } else {
currentNote = 0;
playJumpSound();
      }
    }      if (digitalRead(BUTTON_PIN) ==
LOW) {       resetGame();
    }
return;
  }    if (digitalRead(BUTTON_PIN) == LOW && birdY >
0) {     birdVelocityY = -4.0;     playJumpSound();
  }    birdVelocityY +=
gravity;   birdY +=
birdVelocityY;

  if (birdY > SCREEN_HEIGHT - 1) {
gameOver = true;
  }    obstacleX -=
obstacleSpeed;
```

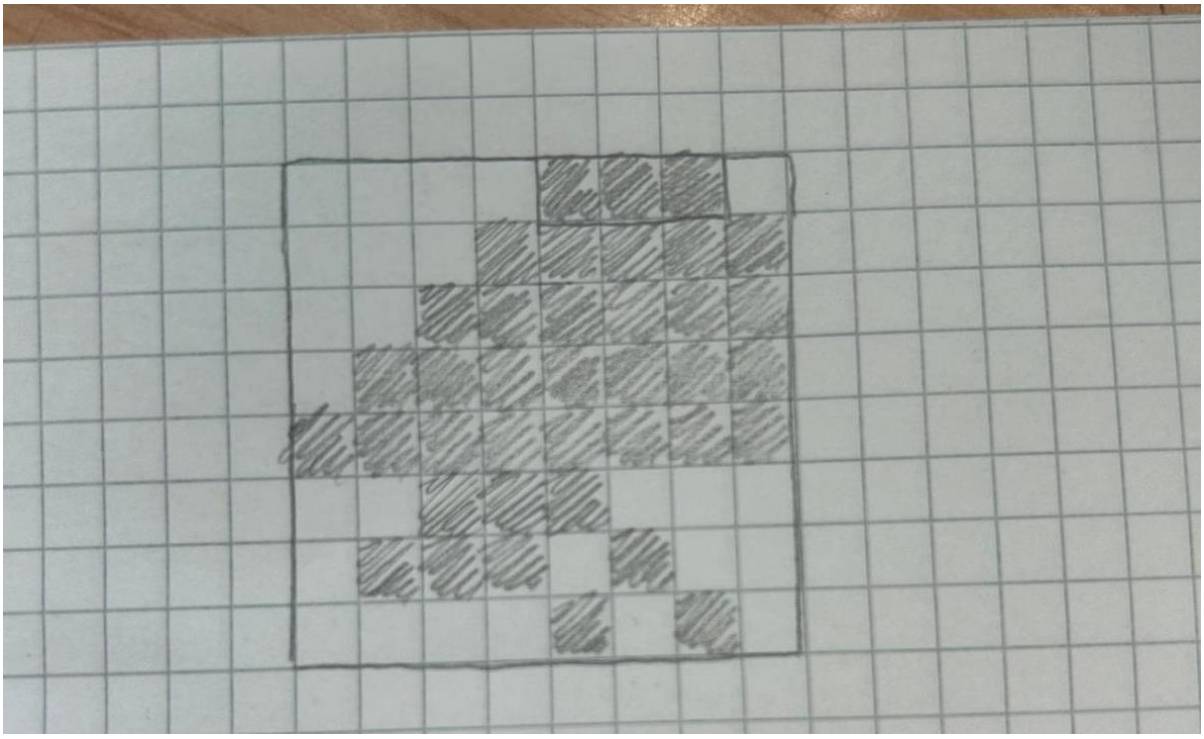
```
  if (obstacleX + obstacleWidth < 0) {
obstacleX = SCREEN_WIDTH;     obstacleGap
= random(15, 30);     score++;
  }    if (birdY < 0 || birdY > SCREEN_HEIGHT
- 1 ||
      (birdY < obstacleGap || birdY > obstacleGap + 20) &&
obstacleX < 20 && obstacleX + obstacleWidth > 0) {     gameOver =
true;
  }
display.clearDisplay();

  display.drawBitmap(20, birdY, bird_bitmap, BIRD_WIDTH, BIRD_HEIGHT,
SSD1306_WHITE);
      display.fillRect(obstacleX,      0,      obstacleWidth,      obstacleGap,
SSD1306_WHITE);   display.fillRect(obstacleX, obstacleGap + 20, obstacleWidth,
SCREEN_HEIGHT - obstacleGap - 20, SSD1306_WHITE);
      display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(SCREEN_WIDTH - 20, 0);
display.print(score);
display.display();   delay(10);
}
void resetGame() {   birdY
= SCREEN_HEIGHT / 2;
birdVelocityY = 0;
obstacleX = SCREEN_WIDTH;
score = 0;   gameOver =
false;   currentNote = 0;
noTone(BUZZER_PIN);
}
void playJumpSound() {
tone(BUZZER_PIN, NOTE_E4, 10);
delay(100);
  noTone(BUZZER_PIN);
}
```

Explicarea codului:

Inițializarea și variabile globale:

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define BUTTON_PIN 4
#define BUZZER_PIN 23

#define BIRD_WIDTH 8
#define BIRD_HEIGHT 8

const uint8_t bird_bitmap[] PROGMEM = {
  0b00001110,
  0b00011111,
  0b00111111,
  0b01111111,
  0b11111111,
  0b00111000,
  0b01110100,
  0b00001010
};

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

int birdY;
float birdVelocityY;
int gravity = 1;
bool gameStarted = false;
bool gameOver = false;
int obstacleX;
int obstacleGap = 20;
int obstacleWidth = 10;
int obstacleSpeed = 2;

int score = 0;
```

- Aici sunt incluse bibliotecile necesare pentru controlul display-ului OLED și pentru grafică.
- Se definesc dimensiunile ecranului, pini pentru buton și buzzer, dimensiunile și grafica păsării.
  Grafica păsării matriceal, arată astfel:



- Un obiect Adafruit_SSD1306 este creat pentru a controla display-ul.
- Variabilele sunt definite pentru gestionarea stării jocului, poziției păsării, vitezei acesteia, gravitației, obstacolelor și scorului.

Melodie și sunete:

```
#define NOTE_C4 262
#define NOTE_E3 165
#define NOTE_G3 196
#define NOTE_E4 330
#define NOTE_G4 392

int melody[] = {
  NOTE_C4, NOTE_E3, NOTE_G3, NOTE_E4,
  NOTE_G4, NOTE_C4, NOTE_E4, NOTE_G4
};

int noteDurations[] = {
  4, 8, 8, 4,
  4, 4, 4, 2
};

int currentNote = 0;
unsigned long noteStartTime = 0;
```

- Se definesc notele și duratele acestora pentru melodia de game over.

Funcția ‘setup()’:

```
void setup() {
  Wire.begin();
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    for (;;);
  }

  display.display();
  delay(2000);
  display.clearDisplay();
  display.display();
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  birdY = SCREEN_HEIGHT / 2;
  birdVelocityY = 0;
  obstacleX = SCREEN_WIDTH;
  pinMode(BUZZER_PIN, OUTPUT);
}
```

- Se inițializează comunicarea I2C și display-ul OLED.

-Se afișează un mesaj de start timp de 2 secunde, apoi ecranul este curățat.

-Pinul butonului este setat ca intrare cu rezistență pull-up.

-Variabilele pentru poziția inițială a păsării și obstacolului sunt inițializate.

-Pinul pentru buzzer este setat ca ieșire.

Funcția ‘loop()’:

```
void loop() {
  if (!gameStarted) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(10, 20);
    display.println("Flappy Bird");
    display.setCursor(5, 45);
    display.println("Press to start");
    display.display();

    if (digitalRead(BUTTON_PIN) == LOW) {
      gameStarted = true;
    }
    return;
  }
```

- Înainte de începerea jocului, se afișează un mesaj și jocul începe când butonul este apăsat.

```
if (gameOver) {
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(10, 20);
  display.println("Game Over");
  display.setCursor(5, 45);
  display.println("Press to play again");
  display.display();

  if (millis() - noteStartTime >= noteDurations[currentNote]) {
    noTone(BUZZER_PIN);
    currentNote++;
    if (currentNote < sizeof(melody) / sizeof(melody[0])) {
      tone(BUZZER_PIN, melody[currentNote]);
      noteStartTime = millis();
    } else {
      currentNote = 0;
      playJumpSound();
    }
  }

  if (digitalRead(BUTTON_PIN) == LOW) {
    resetGame();
  }
  return;
}
```

- Dacă jocul se termină, se afișează un mesaj de game over și se așteaptă apăsarea butonului pentru a începe un nou joc.

```
if (digitalRead(BUTTON_PIN) == LOW && birdY > 0) {
  birdVelocityY = -4.0;
  playJumpSound();
}
```

- Dacă butonul este apăsat și păsarea nu a atins marginea superioară a ecranului, păsarea "sare" și se produce sunetul corespunzător.

```
birdVelocityY += gravity;
birdY += birdVelocityY;

if (birdY > SCREEN_HEIGHT - 1) {
  gameOver = true;
}
```

- Poziția păsării este actualizată în funcție de gravitate. Dacă păsarea atinge partea de jos a ecranului, jocul se termină.

```
  obstacleX -= obstacleSpeed;

  if (obstacleX + obstacleWidth < 0) {
    obstacleX = SCREEN_WIDTH;
    obstacleGap = random(15, 30);
    score++;
  }

  if (birdY < 0 || birdY > SCREEN_HEIGHT - 1 ||
      (birdY < obstacleGap || birdY > obstacleGap + 20) &&
      obstacleX < 20 && obstacleX + obstacleWidth > 0) {
    gameOver = true;
  }
```

- Obstacolele se mișcă spre stânga. Când un obstacol trece complet pe lângă ecran, este reinițializat la o poziție aleatoare și se incrementează scorul.
- Se afișează toate elementele jocului (păsarea, obstacolele, scorul) și se face un mic

```
  display.clearDisplay();


  display.drawBitmap(20, birdY, bird_bitmap, BIRD_WIDTH, BIRD_HEIGHT, SSD1306_WHITE);


  display.fillRect(obstacleX, 0, obstacleWidth, obstacleGap, SSD1306_WHITE);
  display.fillRect(obstacleX, obstacleGap + 20, obstacleWidth, SCREEN_HEIGHT - obstacleGap - 20, SSD1306_WHITE);


  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(SCREEN_WIDTH - 20, 0);
  display.print(score);

  display.display();
  delay(10);
}
```

delay pentru a menține o rată constantă de actualizare a ecranului.

Funcțiile auxiliare și efectele sonore:

```
void resetGame() {
  birdY = SCREEN_HEIGHT / 2;
  birdVelocityY = 0;
  obstacleX = SCREEN_WIDTH;
  score = 0;
  gameOver = false;
  currentNote = 0;
  noTone(BUZZER_PIN);
}

void playJumpSound() {
  tone(BUZZER_PIN, NOTE_E4, 10);
  delay(100);
  noTone(BUZZER_PIN);}
```

- ‘resetGame()’: Resetează variabilele la valorile inițiale pentru un nou joc.
- ‘playJumpSound()’: Emite un sunet scurt când păsarea "sare".

3. Modificarea codului

În cadrul proiectului, codul preluat avea alta abordare pentru modelul de pasăre. Inițial, pasărea era reprezentată printr-un cerc generat de comanda ‘display.drawCircle(20, birdY, 3, SSD1306_WHITE); ’.

Am considerat că această reprezentare poate fi îmbunătățită, așa că am modificat codul pentru a folosi un bitmap mai detaliat pentru pasăre. Am înlocuit cercul cu un bitmap definit de matricea bird_bitmap, astfel încât pasărea are acum o formă și detalii mai complexe.

4. Surse folosite:

[1] - https://www.instructables.com/Flappy-Bird-Game-With-ESP32-and-SH1106-OledDispla/

[2] - https://wokwi.com/projects/379915692831270913